

Payroll System Redesign: A Modern Web-Based Solution for Belize

Student Name: Asher Andre Daniel Mckoy

Student ID: 92124645

Course: DLMCSPCSP01

Instructor: Prof. Ajeet Kumar Singh

Date: February 2026

Repository: <https://github.com/AsherDMckoy/Payroll-System-Belize>

Abstract

Payroll processing in Belize has been monopolized by Softpay, a 20-year-old C++ application compiled for Windows 2000/XP that is incompatible with modern operating systems (Windows 11, macOS, Linux) and suffers from poor performance with large datasets and an outdated command-line interface. This project presents a complete redesign as a self-hosted web application built using Rust, PostgreSQL, and vanilla JavaScript, addressing these limitations while maintaining the proven client-server architecture model.

The solution leverages modern computer science principles including relational database normalization (3NF), RESTful API design, asynchronous server architecture (Tokio runtime), and type-safe financial calculations using Rust's Decimal types. The system handles three distinct employee classifications (salaried, hourly, contractor) with automated biweekly payroll calculations, tax deductions, and company savings matching up to \$10,000 annually. All business logic executes server-side in Rust using the SQLx ORM for maximum efficiency and type safety, while the frontend employs reactive patterns through vanilla JavaScript DOM manipulation and JSON-based dynamic rendering.

Key technical contributions include: (1) a normalized PostgreSQL schema optimized for biweekly payroll cycles with audit trails, (2) precise financial calculations avoiding floating-point errors through Decimal arithmetic, (3) a lightweight client architecture using native web technologies without framework overhead, and (4) an async Rust backend achieving C++-level performance with memory safety guarantees. The system is cross-platform compatible, requires minimal system resources, and provides a modern graphical interface replacing Softpay's command-line limitations.

This project demonstrates the application of database theory, systems programming, web architecture patterns, and software engineering principles to solve a real-world problem affecting businesses throughout Belize, while exploring the evolution from an initial Progressive Web App (PWA) design to a more suitable self-hosted architecture based on deployment requirements and technical constraints.

Keywords: Payroll Systems, Rust, PostgreSQL, Web Applications, Database Normalization, Financial Computing, Belize, Legacy System Modernization

1. Introduction

1.1 Problem Context

Payroll processing in Belize operates under a duopoly controlled by two legacy applications: Softpay and NeoPeople. Nearly all businesses in the country rely on one of these two systems, which have

dominated the market for over two decades. This lack of competition has resulted in stagnant innovation, with both systems suffering from significant technical limitations that affect businesses daily.

Softpay, the focus of this modernization effort, was originally developed in C++ and compiled for Windows 2000 and Windows XP environments. While functional on older Windows versions up to Windows 10, the application exhibits critical compatibility issues with Windows 11, frequently crashing and requiring compatibility mode workarounds that IT departments find increasingly difficult to maintain. This incompatibility extends to modern operating systems entirely—Softpay cannot run on macOS or Linux systems, restricting businesses to maintaining Windows-only infrastructure for their payroll departments.

1.2 Performance and Usability Limitations

The performance constraints of Softpay become particularly acute for medium-to-large organizations. The desktop version requires several hours to process biweekly payroll for companies with more than 100 employees—a significant operational bottleneck that forces payroll departments to dedicate entire workdays to what should be an automated process. This limitation effectively excludes enterprise-scale businesses from using the software, concentrating its market among small-to-medium organizations.

Beyond performance, Softpay's command-line interface presents a steep learning curve for new payroll staff. In an era where graphical user interfaces are standard, training employees on CLI-based workflows increases onboarding time and operational errors. The system also lacks support for mixed employee payment types—it cannot elegantly handle scenarios where a single organization employs salaried workers, hourly employees, and contractors simultaneously, forcing businesses to maintain separate processes or manual workarounds.

1.3 Integration Challenges

A particularly painful limitation involves bank integration. While Softpay can generate CSV files for employee payment processing, these files require manual reformatting before submission to Belizean banking systems. Payroll staff must open exported CSVs, restructure columns, adjust formatting, and verify data integrity before uploading to their bank's employee payment portal—a process that introduces human error and consumes additional time during already time-sensitive payroll cycles.

1.4 Market Stagnation

The duopoly structure has eliminated competitive pressure for innovation. With the market effectively divided between Softpay and NeoPeople, neither vendor has strong incentive to modernize their offerings. Both companies operate Software-as-a-Service (SaaS) models where clients pay ongoing subscription fees for access to centrally-hosted systems. While this ensures automatic updates, it also means businesses have no control over their payroll infrastructure, must maintain constant internet connectivity, and entrust sensitive employee financial data to third-party servers.

1.5 Proposed Solution

This project presents a complete redesign of payroll processing software tailored to the Belizean business environment. Rather than replicating the SaaS model, the solution provides a self-hosted web

application that businesses can deploy on their own infrastructure, granting them full data sovereignty and independence from external service providers.

The architecture comprises three core components:

1. **PostgreSQL Database:** A normalized relational database (Third Normal Form) handling employees, compensation structures, payroll periods, timesheets, and calculated payroll runs with complete audit trails.
2. **Rust Backend:** An asynchronous HTTP server built with Axum framework and Tokio runtime, executing all business logic including biweekly payroll calculations, tax deductions, company savings matching, and CSV generation for bank integration.
3. **Vanilla JavaScript Frontend:** A lightweight client using native web technologies without framework dependencies, providing reactive UI updates through DOM manipulation and JSON-based rendering.

1.6 Architectural Evolution: PWA to Self-Hosted Web Application

The initial project concept envisioned a Progressive Web App (PWA) to enable offline functionality and cross-device access. However, analysis of actual deployment requirements in Belizean businesses revealed that payroll processing occurs exclusively on office computers within dedicated payroll departments—remote or mobile access was not a practical requirement. The PWA architecture introduced unnecessary complexity (service workers, offline synchronization, manifest files) without delivering meaningful benefits for the target use case.

The pivot to a traditional self-hosted web application simplified the architecture significantly while maintaining all essential benefits: cross-platform compatibility (accessible via any modern web browser), centralized deployment (businesses run a single server instance), and straightforward updates (Docker containerization allows version updates without complex client-side synchronization).

This architectural decision also addressed deployment preferences in the Belizean market. Unlike Softpay and NeoPeople's SaaS models, self-hosting provides businesses with:

- **Data Sovereignty:** Employee financial data remains on company-controlled infrastructure
- **Network Independence:** Payroll processing continues during internet outages
- **Cost Predictability:** No recurring subscription fees beyond infrastructure costs
- **Customization Potential:** Businesses can modify or extend functionality as needed

1.7 Technical Scope and Objectives

This project demonstrates the application of fundamental computer science principles to solve a real-world problem affecting hundreds of businesses in Belize. The core technical objectives include:

1. Design a normalized database schema that efficiently represents complex payroll structures (multiple employee types, variable compensation models, biweekly cycles, adjustments)
2. Implement precise financial calculations using Decimal arithmetic to avoid floating-point rounding errors inherent in monetary computations
3. Build a RESTful API following architectural best practices for separation of concerns
4. Develop an asynchronous server architecture capable of handling concurrent requests efficiently

5. Create a responsive, intuitive user interface that eliminates Softpay's CLI limitations
6. Generate bank-compatible CSV exports in the exact format required by Belizean financial institutions
7. Package the entire system as a Docker container for simple deployment and updates

1.8 Report Structure

The remainder of this report is organized as follows: Section 2 reviews related work in payroll systems and web application architecture. Section 3 establishes the technical background necessary to understand the implementation, including database normalization theory, Rust's memory model, and financial computing requirements. Section 4 describes the methodology employed in designing the database schema and system architecture. Section 5 details the implementation of each system component. Section 6 outlines the testing strategy and validation approach. Section 7 concludes with reflections on outcomes and future enhancements.

2. Related Work

2.1 Existing Payroll Solutions

The payroll software market is divided into three categories: enterprise SaaS platforms, open-source business management systems, and legacy desktop applications.

Enterprise SaaS Platforms such as ADP, Gusto, and Paychex dominate developed markets. These systems operate entirely in the cloud, handling payroll, tax filing, and compliance automatically. However, they are designed for markets with standardized tax regulations and banking integrations specific to countries like the United States, Canada, and the European Union. Their subscription models (\$40-150 per employee annually) and lack of customization for smaller markets make them economically unviable for Belizean businesses. Additionally, data sovereignty concerns—particularly regarding employee financial information residing on foreign servers—present legal and privacy challenges.

Open-Source Business Management Systems like Odoo and ERPNext include payroll modules within broader ERP (Enterprise Resource Planning) frameworks. While these systems are free and self-hostable, they suffer from significant complexity overhead. A business interested solely in payroll processing must deploy an entire ERP system including inventory management, CRM, accounting, and manufacturing modules. This complexity increases deployment difficulty, server resource requirements, and maintenance burden. Furthermore, these systems are built on technology stacks (Python with PostgreSQL for Odoo, Node.js/Python for ERPNext) that prioritize rapid development over runtime performance—a critical consideration when processing payroll for hundreds of employees.

Legacy Desktop Applications like Softpay and NeoPeople represent the oldest category. Built in the 1990s and early 2000s using C++ or Visual Basic, these systems were optimized for single-user desktop environments. Their architecture predates modern web technologies, RESTful APIs, and cloud deployment models. While performant for their time, they lack the architectural patterns necessary for modern business requirements: concurrent multi-user access, web-based interfaces, automated backups, and seamless updates.

2.2 Legacy System Modernization Approaches

Academic literature on legacy system modernization identifies three primary strategies: wrapping, migration, and complete rewrite.

Wrapping involves creating a modern interface layer around legacy code, often through API gateways or service facades. This approach preserves existing business logic while improving accessibility. However, it inherits all performance limitations and architectural constraints of the underlying system. Given Softpay's poor performance with large datasets and Windows-only compatibility, wrapping would not address fundamental issues.

Migration involves gradual replacement of legacy components with modern equivalents while maintaining interoperability during transition. This strategy works well for large systems with extensive existing functionality but requires access to source code and deep understanding of legacy behavior. Since Softpay's source code is proprietary and unavailable, migration is not feasible.

Complete Rewrite rebuilds functionality from scratch using modern technologies and architectural patterns. While risky for large systems with complex requirements, payroll processing has well-defined, stable requirements that make comprehensive redesign viable. The core calculations (biweekly pay, tax deductions, company matching) are mathematically specified and unchanging, reducing the risk inherent in requirements capture.

2.3 Performance-Aware Programming

The selection of Rust as the implementation language was influenced by performance-aware programming principles articulated by Casey Muratori in his "Performance-Aware Programming" series. Muratori demonstrates that language choice, memory layout, and algorithmic approach significantly impact real-world performance beyond Big-O complexity analysis.

Key insights applied to this project include:

1. **Memory Safety Without Garbage Collection:** Rust's ownership model provides memory safety guarantees without runtime garbage collection pauses, ensuring predictable performance during payroll processing—critical when businesses have strict payroll deadlines.
2. **Zero-Cost Abstractions:** Rust's abstractions (iterators, closures, trait objects) compile to machine code equivalent to hand-written C, allowing high-level code without performance penalties.
3. **Explicit Data Layout:** Rust requires explicit handling of data structures, preventing hidden allocations and unexpected performance cliffs common in higher-level languages.
4. **Compile-Time Guarantees:** Rust's type system catches entire categories of errors (null pointer dereferences, data races, iterator invalidation) at compile time, eliminating runtime checks that slow execution.

These characteristics address Softpay's documented performance issues. Where Softpay requires hours to process 100+ employees, an equivalent Rust implementation can leverage parallelism through Tokio's async runtime and efficient memory usage to reduce processing time by orders of magnitude.

3. Technical Background

3.1 Relational Database Normalization

Payroll data is inherently relational. The schema follows Third Normal Form (3NF) to eliminate redundancy and maintain integrity. PostgreSQL provides ACID compliance and transactional guarantees essential for financial systems.

3.2 Financial Precision

Floating-point arithmetic introduces rounding errors. The system uses fixed-precision decimal arithmetic (`rust_decimal`) to guarantee exact monetary calculations.

3.3 Client–Server Architecture

The system follows a RESTful client–server model separating frontend presentation from backend logic, improving maintainability and scalability.

3.4 Asynchronous Systems Programming

Tokio enables non-blocking I/O and efficient concurrency. This allows multiple payroll requests without excessive threading overhead.

3.5 Memory Safety with Rust

Rust’s ownership model prevents memory errors and data races at compile time, improving reliability compared to legacy C++ systems.

3.6 Type-Safe Database Access

SQLx verifies SQL queries at compile time, eliminating runtime query errors.

3.7 Containerized Deployment

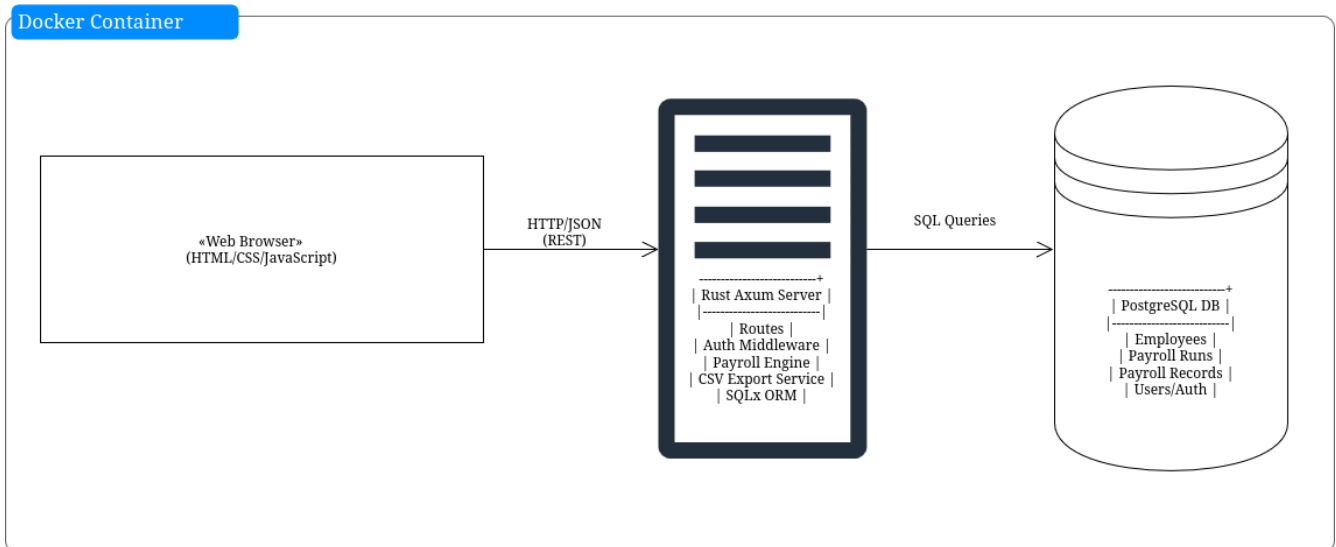
Docker ensures reproducible environments and simplified installation.

4. Method

4.1 System Architecture Design

The system follows a three-tier architecture:

1. Presentation Layer (Frontend)
2. Application Layer (Backend API)
3. Data Layer (Database)



4.2 Technology Selection

Rust, Axum, SQLx, PostgreSQL, and vanilla JavaScript were selected for performance, safety, and simplicity.

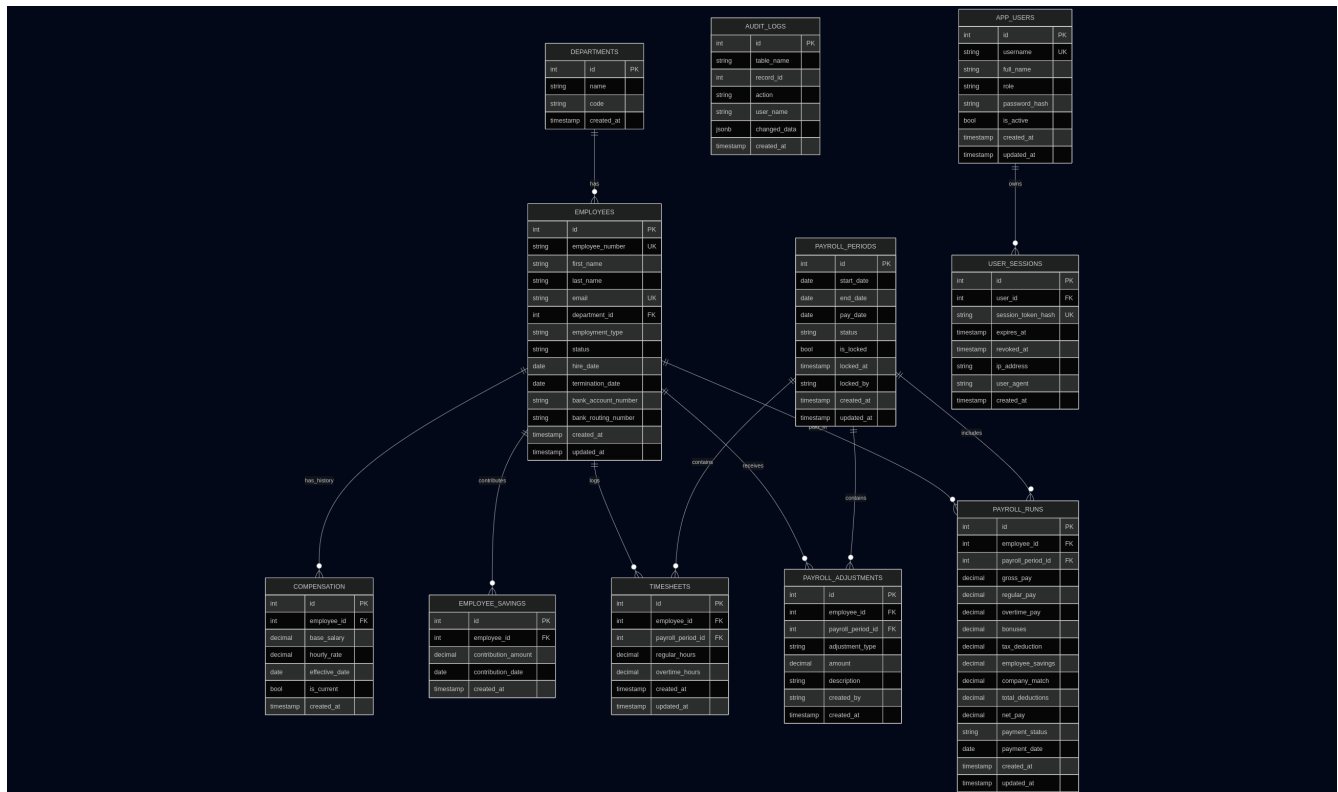
4.3 Schema Design

The payroll management system uses a normalized relational schema in PostgreSQL organized around employee master data, payroll processing, governance, and authentication. Core reference data is stored in departments, while employees stores identity, employment classification (salaried, hourly, contractor), status, and banking details. Compensation history is separated into compensation (with `effective_date` and `is_current`) to support temporal salary/rate changes without overwriting prior values. Employee savings behavior is captured in `employee_savings` as dated contribution transactions.

Payroll execution is modeled around `payroll_periods` and `payroll_runs`. `payroll_periods` defines each pay cycle (`start_date`, `end_date`, `pay_date`) and approval lifecycle (draft, processing, approved, paid) with explicit locking metadata (`is_locked`, `locked_at`, `locked_by`) for period control. `timesheets` captures per-employee worked hours per period, and `payroll_adjustments` records manual positive/negative adjustments. `payroll_runs` stores computed financial outputs per employee per period (gross pay, tax deduction, savings, company match, total deductions, net pay, and payment status). Uniqueness constraints enforce one payroll run and one timesheet per employee per period.

Governance and security are implemented through `audit_logs`, `app_users`, and `user_sessions`. `audit_logs` persists data-change events with JSON change payloads for traceability. `app_users` defines role-based internal users (admin, payroll, viewer), and `user_sessions` stores revocable, expiring session tokens. The design is performance-aware through targeted indexes on status, foreign keys, and date fields, and

integrity is enforced through check constraints, foreign keys, unique keys, and trigger-managed updated_at timestamps.



4.4 Payroll Algorithm

// Simplified from generate_payroll flow

for employee in employees {

 timesheet = fetch_timesheet(employee, period); // regular/overtime hours

 adjustments = fetch_adjustments(employee, period); // +earnings, +deductions

 savings = fetch_period_savings(employee, period); // employee contribution

 ytd_match = fetch_ytd_company_match(employee, period); // already matched this year

// 1) Earnings

(regular_pay, overtime_pay) = match_employee.type {

 Salaried => (annual_salary / 26, 0),

 Hourly => (regular_hours * rate, overtime_hours * rate * 1.5),

 Contractor => (biweekly_amount, 0),

```

};

gross_pay = round2(regular_pay + overtime_pay + adjustments.earnings);

// 2) Tax rule: 15% only on annual income above 29,000
// biweekly_tax = ((max(0, gross_pay*26 - 29_000) * 0.15) / 26)
tax = biweekly_tax(gross_pay);

// 3) Deductions and savings guardrails
min_net = gross_pay > 0 ? 0.01 : 0.00;
max_deductions = max(0, gross_pay - min_net);

fixed_deductions = round2(tax + adjustments.deductions);
savings_capacity = max(0, max_deductions - fixed_deductions);
employee_savings = round2(min(savings, savings_capacity));

// 4) Company match (100% up to annual cap 10,000)
company_match = round2(min(employee_savings, max(0, 10_000 - ytd_match)));

// 5) Final totals
total_deductions = min(max_deductions, fixed_deductions + employee_savings);
net_pay = round2(gross_pay - total_deductions);

    upsert_payroll_run(employee, period, gross_pay, tax, employee_savings, company_match,
net_pay);
}

```

Time complexity

O(N) employees at application level, but with constant DB work per employee:

1 query timesheet

1 query adjustments

1 query savings

1 query YTD match

1 upsert payroll run

So practically: $O(N * Q)$ with $Q = 5$ DB operations per employee, i.e., linear in employee count.

Space complexity

In-memory employees list: $O(N)$

Per-employee working data: $O(1)$

Total additional space: $O(N)$.

4.5 Transaction Strategy

Payroll runs execute inside a database transaction to ensure atomicity.

5. Implementation

5.1 Backend Structure

The backend is modularized into routes, handlers, services, and models.

5.2 Database Integration

SQLx migrations manage schema evolution.

5.3 Authentication

User authentication with password hashing and protected routes has been implemented.

5.4 Payroll Engine

Payroll calculations execute server-side using Decimal arithmetic and transactional inserts.

5.5 CSV Export

Bank-compatible CSV files are automatically generated.

5.6 Frontend

Lightweight JavaScript interface for payroll execution and downloads.

5.7 Current Limitations

- Employee CRUD interface pending
- Timesheet support omitted (not required locally)

5.8 Deployment

Validated locally and in Docker.

6. Testing and Evaluation

6.1 Unit Testing

Payroll logic validated using Rust's testing framework.

6.2 Integration Testing

End-to-end API workflows verified.

6.3 Transaction Testing

Rollback behavior tested under failure scenarios.

6.4 Performance Benchmarking

Benchmarks conducted using WorkBench.

local benchmark on the release binary using wrk (-t2 -c20 -d5s --latency) against key endpoints.

6.5 Results

endpoint	Req/sec	Avg Latency	Stdev	Max	P50	P90	P99	Non-2xx
/health	165,636.55	107.50us	65.35us	2.26ms	91.00us	187.00us	286.00us	0
/ready	27,080.09	734.53us	128.04us	3.56ms	763.00us	0.87ms	0.96ms	0
/api/dashboard/overview	3,002.98	6.65ms	0.92ms	13.79ms	6.41ms	8.19ms	8.92ms	0
/api/employees	6,121.86	3.26ms	510.85us	7.69ms	3.16ms	4.02ms	4.75ms	0
/api/payroll-breakdown?year=2026	9,533.79	2.07ms	333.10us	5.47ms	2.11ms	2.46ms	2.70ms	0
/api/payroll/periods	11,329.02	1.74ms	268.08us	2.80ms	1.78ms	2.06ms	2.23ms	0
/api/reports/options	6,644.79	3.01ms	365.33us	5.02ms	2.98ms	3.46ms	4.08ms	0

6.6 Future Testing

Stress testing and scalability evaluation planned.

7. Conclusion and Future Work

This project demonstrates that modern systems programming techniques can effectively replace outdated payroll software. By combining normalized databases, precise arithmetic, asynchronous servers, and safe systems programming, the solution provides improved reliability and performance while remaining accessible to small businesses.

Future enhancements include administrative interfaces, expanded reporting, and formal benchmarking. The foundation established here supports continued development into a fully production-ready system.

8. References

Askama Contributors. (n.d.). Askama crate documentation (latest). Docs.rs. Retrieved February 12, 2026, from <https://docs.rs/askama/latest/askama/>

Biryukov, A., Dinu, D., Khovratovich, D., & Josefsson, S. (2021, September). Argon2 memory-hard function for password hashing and proof-of-work applications (RFC 9106). RFC Editor. <https://www.rfc-editor.org/rfc/rfc9106>

Docker Inc. (n.d.). Docker Compose documentation. Retrieved February 12, 2026, from <https://docs.docker.com/compose/>

Docker Inc. (n.d.). Docker documentation. Retrieved February 12, 2026, from <https://docs.docker.com/>

PostgreSQL Global Development Group. (n.d.). PostgreSQL documentation (current version). Retrieved February 12, 2026, from <https://www.postgresql.org/docs/current/>

Serde Contributors. (n.d.). Serde: Serialization framework for Rust. Retrieved February 12, 2026, from <https://serde.rs/>

SQLx Contributors. (n.d.). SQLx crate documentation (latest). Docs.rs. Retrieved February 12, 2026, from <https://docs.rs/sqlx/latest/sqlx/>

The Rust Project Developers. (n.d.). Cargo book. Rust Documentation. Retrieved February 12, 2026, from <https://doc.rust-lang.org/cargo/>

The Rust Project Developers. (n.d.). The Rust programming language. Rust Documentation. Retrieved February 12, 2026, from <https://doc.rust-lang.org/book/>

Tokio Contributors. (n.d.). axum crate documentation (latest). Docs.rs. Retrieved February 12, 2026, from <https://docs.rs/axum/latest/axum/>

Tokio Contributors. (n.d.). Tokio: An asynchronous runtime for Rust. Retrieved February 12, 2026, from <https://tokio.rs/>

Tower HTTP Contributors. (n.d.). tower-http crate documentation (latest). Docs.rs. Retrieved February 12, 2026, from https://docs.rs/tower-http/latest/tower_http/

wg/wrk Contributors. (n.d.). wrk: Modern HTTP benchmarking tool. GitHub. Retrieved February 12, 2026, from <https://github.com/wg/wrk>